

Healthcare AI QA Engineering Platform

Step 1.12 Implementation Report

Project Name:
CareFlow Health

Step:
STEP 1.12 - CareFlow Health Frontend Application Implementation

Objective:
Implement a production-like frontend-only healthcare application aligned with approved BRD, PRD, UI architecture, and testability standards.

Architecture Summary:

- React + TypeScript + Vite + React Router runtime foundation.
- Application shell layout with role indicator, navigation, and routed content.
- Role-aware route and action control via deterministic UI simulation.
- Reusable common components for page titles, state panels, status badges, and confirmations.
- Centralized mock service and mock data contracts for future API swap.

Implementation Summary:

- Frontend scaffolded and configured as strict TypeScript Vite application.
- Role-based login simulation and guarded routing implemented.
- Core modules implemented: dashboard, patients, appointments, providers, records, prescriptions, billing and insurance, claims, notifications, reports, admin, profile, settings, help, search, unauthorized and not-found pages.
- Deterministic in-memory data and mutation flows implemented for realistic behavior.
- Automation-friendly data-testid selectors implemented for major workflows.

Technologies:

- React
- TypeScript
- Vite
- React Router
- OXlint

Pages and Modules:

- Login
- Dashboard
- Patient Management, Search, Detail
- Appointments, Appointment Detail
- Providers
- Medical Records
- Prescriptions
- Billing and Insurance
- Claims

- Notifications
- Reports
- Administration
- Profile, Settings, Help, Search

Validation:

- npm run lint: PASS
- npm run build: PASS

Testability:

- Stable deterministic selectors implemented with data-testid.
- Deterministic IDs and predictable state transitions for automation reliability.

AI Boundary:

- No runtime RAG, MCP, tool-calling, agentic workflow, or LLM integration in this step.
- UI architecture preserves clean boundary points for planned AI integrations.

AI Concepts Status in Step 1.12:

- IMPLEMENTED: none (runtime AI intentionally deferred)
- PLANNED: RAG, RAGAS, observability, MCP, tool-calling, LangGraph, multi-agent, HITL, guardrails, prompt-injection protection, AI security, LLMOps, AI quality gates, AI failure analysis, AI defect management, evaluation datasets/runs, model and prompt version tracking, experiment tracking.

Risks and Known Issues:

- TypeScript build metadata file writes required unsandboxed build execution in this environment.
- Backend/API security controls are intentionally out of scope for this frontend-only step.

Interview Summary:

This implementation demonstrates enterprise frontend architecture, role-aware UX behavior, validation discipline, accessibility fundamentals, deterministic testability design, and readiness for later Playwright automation plus future AI integration layers.